A TO Z MASTER GUIDE

Python Programming

Basics to Advanced - Step by Step - Projects - Interview Questions

Prepared specially for Koushik
Beginner-friendly 2026 edition

Inside: Core Python | Collections | OOP | Files | Data Tools | Interview

How to use	Oka topic chadivi code ni own-ga type cheyyi. First output guess cheyyi, tarvatha run cheyyi. Interview answer boxes ni loud-ga 2 times practice cheyyi.

Table of Contents

How to Study This Guide . . . 5

Daily study method - 60 to 90 minutes . . . 5

Interview lo questions ela untayi? . . . 5

1. Python Foundation and Setup . . . 5

1.1 What is Python? . . . 5

1.2 Install and run on Windows . . . 5

1.3 How Python runs . . . 6

2. Variables, Data Types and I/O . . . 6

2.1 Variables and naming . . . 6

2.2 Built-in type overview . . . 6

2.3 Input, conversion and output . . . 6

2.4 Mutable vs immutable . . . 7

3. Operators and Expressions . . . 7

3.1 == versus is . . . 7

4. Conditions and Pattern Matching . . . 7

4.1 if, elif, else . . . 7

4.2 Conditional expression . . . 7

4.3 match-case (Python 3.10+) . . . 8

5. Loops . . . 8

5.1 for and range . . . 8

5.2 while, break, continue and loop else . . . 8

5.3 Core loop programs . . . 8

6. Strings . . . 9

6.1 Indexing, slicing and methods . . . 9

6.2 split, join and cleanup . . . 9

6.3 Palindrome and frequency . . . 9

7. Lists, Tuples, Sets and Dictionaries . . . 9

7.1 List . . . 9

7.2 Tuple . . . 9

7.3 Set . . . 9

7.4 Dictionary . . . 9

7.5 Comprehensions . . . 10

8. Functions . . . 10

8.1 Definition, return and docstring . . . 10
8.2 Argument types . . . 10
8.3 Mutable default argument trap . . . 10
8.4 Scope and LEGB . . . 11
8.5 Lambda, map, filter and reduce . . . 11

9. Modules, Packages and Environments . . . 11
9.1 Imports . . . 11
9.2 Main guard . . . 11
9.3 Virtual environment and pip . . . 11

10. Exceptions and Debugging . . . 12
10.1 try, except, else, finally . . . 12
10.2 Raise and custom exception . . . 12
10.3 Debug checklist . . . 12

11. File Handling, CSV and JSON . . . 12
11.1 Text files with context manager . . . 12
11.2 CSV . . . 12
11.3 JSON . . . 13

12. Object-Oriented Programming . . . 13
12.1 Class, object and constructor . . . 13
12.2 Inheritance and super . . . 13
12.3 Class method and static method . . . 14

13. Iterables, Iterators and Generators . . . 14
13.1 Iterator protocol . . . 14
13.2 Generator . . . 14
13.3 Generator expression . . . 14

14. Decorators and Context Managers . . . 15
14.1 Function decorator . . . 15
14.2 Custom context manager . . . 15

15. Useful Standard Library Tools . . . 15
15.1 Dates . . . 15
15.2 Regular expressions . . . 16
15.3 SQLite . . . 16

16. Testing, Style and Type Hints . . . 16
16.1 Assertions and unittest . . . 16
16.2 Clean-code checklist . . . 16

17. Data Analysis Starter: NumPy and pandas . . . 17

17.1 Install . . . 17
17.2 NumPy array . . . 17
17.3 pandas DataFrame . . . 17
17.4 Missing values and filtering . . . 17

18. Advanced Concepts You Should Know . . . 17

18.1 Shallow vs deep copy . . . 17
18.2 Memory management and garbage collection . . . 17
18.3 GIL . . . 18
18.4 Threading, multiprocessing and async . . . 18
18.5 Time complexity quick view . . . 18

19. Mini Projects . . . 18

19.1 Number guessing game . . . 18
19.2 Expense tracker . . . 18
19.3 Project ideas . . . 19

20. Python Interview Questions with Model Answers . . . 19

21. Output and Coding Interview Practice . . . 23

21.1 Output questions . . . 23
21.2 Most asked coding tasks . . . 24

22. Practice Sets and 30-Day Plan . . . 24

22.1 Beginner practice . . . 24
22.2 Intermediate practice . . . 25
22.3 30-day schedule . . . 25

23. Quick Revision Cheat Sheet . . . 25

How to Study This Guide

Ee PDF Python ni zero nunchi interview level varaku revise cheyyadaniki design chesanu. Prathi lesson lo concept, syntax, example, output, explanation, mistakes untayi. Chusi copy cheyyadam kanna own-ga type cheyyadam important.

Daily study method - 60 to 90 minutes

Time	Work	Reason
10 min	Yesterday notes recall	Memory strong avutundi
20 min	One new concept	Overload avvakunda concept clear avutundi
25 min	Examples own-ga type and run	Syntax and logic practice
15 min	2 coding questions	Problem solving improve avutundi
10 min	3 interview answers loud-ga cheppu	Communication confidence vastundi

Golden rule	Code chadivithe telisinattu anipistundi; kani blank screen meeda type chesthe ne nijamga vachindo telustundi.

Interview lo questions ela untayi?

- Definition: What is a pointer / list / class?
- Difference: array vs pointer, list vs tuple, compiler vs interpreter.
- Output-based: small code ichi output enti ani adugutaru.
- Debugging: error unna code ni correct cheyyamani adugutaru.
- Coding: reverse, palindrome, prime, array/string problems.
- Project: nuvvu rasina code lo decisions enduku teesukunnavo adugutaru.

Answer formula	1) One-line definition. 2) Two important points. 3) Small example. 4) Asked only if needed, limitation. Ila chepthe answer neat-ga untundi.

1. Python Foundation and Setup

1.1 What is Python?

Python is a high-level, general-purpose language known for readable syntax and a large ecosystem. It is dynamically typed and commonly implemented using an interpreter and bytecode virtual machine. It is used in automation, web development, data analysis, AI, testing and scripting.

Easy Telugu	Python syntax simple-ga untundi. Type declarations takkuva, indentation important. Fast-ga code rayadaniki and data analysis ki chala useful.

1.2 Install and run on Windows

- Download Python 3 from python.org and tick Add Python to PATH during installation.
- Check: `python --version` or Windows lo `py --version`.
- Create file `hello.py` in VS Code or IDLE.
- Run: `python hello.py` or `py hello.py`.

- Interactive mode: type `python`, test expressions, exit with `exit()`.

First Python program - hello.py

```
print("Hello, Koushik!")
name = input("Your name: ")
print(f"Welcome, {name}!")
```

Indentation	Python lo `{}` badulu indentation blocks ni define chestundi. Usually 4 spaces use cheyyi; tabs and spaces mix cheyyaku.

1.3 How Python runs

Source code is parsed and typically compiled to bytecode, then a Python virtual machine executes it. Details vary by implementation; CPython is the most common implementation.

2. Variables, Data Types and I/O

2.1 Variables and naming

```
age = 22
price = 199.50
name = "Koushik"
is_learning = True
nothing = None
print(type(age), type(name))
```

A variable name is bound to an object. Python determines the current object type at runtime. Rebinding the same name to another type is allowed, but consistent names make code clearer.

2.2 Built-in type overview

Category	Types	Example
Numeric	int, float, complex	42, 3.14, 2+3j
Boolean	bool	True, False
Text	str	"hello"
Sequence	list, tuple, range	[1,2], (1,2), range(5)
Set	set, frozenset	{1,2,3}
Mapping	dict	{"name": "Koushik"}
Binary	bytes, bytearray	b"ABC"
No value	NoneType	None

2.3 Input, conversion and output

```
age_text = input("Enter age: ")   # input always returns str
age = int(age_text)
height = float(input("Enter height: "))
print("Age:", age)
print(f"Height: {height:.1f}")
```

Common error	`input()` string return chestundi. `input() + 5` TypeError istundi. First `int(input())` or `float(input())` convert cheyyi.

2.4 Mutable vs immutable

Mostly immutable	Mutable
int, float, bool, str, tuple, frozenset, bytes	list, dict, set, bytearray
Operation usually creates a new object	Contents can change in place

3. Operators and Expressions

Family	Operators / examples
Arithmetic	+, -, *, /, //, %, **
Comparison	==, !=, <, <=, >, >=
Logical	and, or, not
Assignment	=, +=, -=, *=, :=
Membership	in, not in
Identity	is, is not
Bitwise	&, |, ^, ~, <<, >>

```
print(7 / 2)    # 3.5 true division
print(7 // 2)   # 3 floor division
print(7 % 2)    # 1 remainder
print(2 ** 5)   # 32 power

name = "Koushik"
print("Kou" in name)  # True
```

3.1 == versus is

`==` compares values. `is` compares object identity. Use `is None` and `is not None`; do not use `is` as a general replacement for equality.

```
a = [1, 2]
b = [1, 2]
print(a == b)  # True: equal contents
print(a is b)  # False: different objects
```

4. Conditions and Pattern Matching

4.1 if, elif, else

```
marks = int(input("Marks: "))
if not 0 <= marks <= 100:
    print("Invalid")
elif marks >= 90:
    print("Grade A")
elif marks >= 75:
    print("Grade B")
elif marks >= 60:
    print("Grade C")
else:
    print("Needs improvement")
```

Python supports chained comparisons: `0 <= marks <= 100`. Empty strings/collections, zero and None are falsy; most other objects are truthy.

4.2 Conditional expression

```
age = 20
status = "Adult" if age >= 18 else "Minor"
```

4.3 match-case (Python 3.10+)

```
command = "start"
match command:
    case "start":
        print("Starting")
    case "stop":
        print("Stopping")
    case _:
        print("Unknown")
```

5. Loops

5.1 for and range

```
for i in range(1, 6):
    print(i, end=" ")

for index, value in enumerate([10, 20, 30], start=1):
    print(index, value)
```

`range(start, stop, step)` excludes stop. Python for loop iterates over any iterable, not only numbers.

5.2 while, break, continue and loop else

```
n = 5
while n > 0:
    n -= 1
    if n == 3:
        continue
    if n == 1:
        break
    print(n)
else:
    print("Runs only if loop ended without break")
```

5.3 Core loop programs

Factorial

```
n = int(input("Number: "))
factorial = 1
for value in range(2, n + 1):
    factorial *= value
print(factorial)
```

Prime check

```
n = int(input("Number: "))
is_prime = n >= 2
divisor = 2
while divisor * divisor <= n and is_prime:
    if n % divisor == 0:
        is_prime = False
    divisor += 1
print("Prime" if is_prime else "Not prime")
```

Dry run	Loop question lo each iteration ki variable values paper meeda rayi. Output question lo mind calculation kanna table safer.

6. Strings

6.1 Indexing, slicing and methods

```
text = "Python"
print(text[0])       # P
print(text[-1])      # n
print(text[1:4])     # yth
print(text[::-1])    # nohtyP
print(text.lower())
print(text.replace("Py", "My"))
print(text.startswith("Py"))
```

Strings are immutable. A method such as replace returns a new string; it does not modify the original.

6.2 split, join and cleanup

```
raw = "  C, Python, SQL  "
items = [item.strip() for item in raw.split(",")]
result = " | ".join(items)
print(result)  # C | Python | SQL
```

6.3 Palindrome and frequency

```
text = input("Text: ")
clean = "".join(ch.lower() for ch in text if ch.isalnum())
print("Palindrome" if clean == clean[::-1] else "Not palindrome")
```

```
from collections import Counter
text = "banana"
counts = Counter(text)
print(counts)
print(counts.most_common(1))
```

7. Lists, Tuples, Sets and Dictionaries

7.1 List

```
marks = [78, 92, 65]
marks.append(88)
marks.extend([70, 81])
marks.insert(1, 99)
removed = marks.pop()
marks.sort(reverse=True)
print(marks, removed)
```

Copy trap	`b = a` same list ki second name. Independent shallow copy kosam `b = a.copy()` or `a[:]`. Nested objects still shared avvachu; deep copy separate concept.

7.2 Tuple

```
point = (10, 20)
x, y = point
single = (5,)  # comma makes one-item tuple
print(x, y)
```

Tuple is immutable and often used for fixed records or multiple return values.

7.3 Set

```
a = {1, 2, 3}
b = {3, 4, 5}
print(a | b)  # union
print(a & b)  # intersection
print(a - b)  # difference
```

7.4 Dictionary

```
student = {"name": "Koushik", "marks": 86}
student["course"] = "Data Analytics"
print(student.get("phone", "Not available"))
for key, value in student.items():
    print(key, value)
```

Collection	Ordered?	Mutable?	Duplicates / lookup
list	Yes	Yes	Duplicates; index lookup
tuple	Yes	No	Duplicates; index lookup
set	No positional order	Yes	Unique; fast membership average
dict	Insertion order	Yes	Unique hashable keys; key lookup

7.5 Comprehensions

```
squares = [n * n for n in range(1, 6)]
evens = {n for n in range(10) if n % 2 == 0}
lengths = {word: len(word) for word in ["C", "Python", "SQL"]}
```

Readable first	Comprehension one simple expression/filter ki baguntundi. Multiple nested conditions unte normal loop clearer.

8. Functions

8.1 Definition, return and docstring

```
def calculate_total(price: float, quantity: int = 1) -> float:
    """Return price multiplied by quantity."""
    return price * quantity

amount = calculate_total(99.5, quantity=3)
print(amount)
```

8.2 Argument types

```
def demo(a, b=10, *args, scale=1, **kwargs):
    print(a, b, args, scale, kwargs)

demo(1, 2, 3, 4, scale=5, active=True)
```

Type	Meaning
Positional	Matched by position
Keyword	Matched by parameter name
Default	Used when argument is omitted
*args	Extra positional arguments as tuple
**kwargs	Extra keyword arguments as dict

8.3 Mutable default argument trap

```
def add_item(item, items=None):
    if items is None:
        items = []
    items.append(item)
    return items
```

Do not use `items=[]` as a default when a new list is expected per call. Defaults are evaluated once when function is defined.

8.4 Scope and LEGB

Python resolves names in Local, Enclosing, Global and Built-in scopes. Use `global` or `nonlocal` only when truly needed; returning values is often clearer.

8.5 Lambda, map, filter and reduce

```
from functools import reduce
nums = [1, 2, 3, 4]
squares = list(map(lambda x: x * x, nums))
evens = list(filter(lambda x: x % 2 == 0, nums))
total = reduce(lambda a, b: a + b, nums, 0)
```

Pythonic note	Simple transform/filter ki list comprehension usually map/filter kanna readable: `[x*x for x in nums]`.

9. Modules, Packages and Environments

9.1 Imports

```
import math
from pathlib import Path
import statistics as stats

print(math.sqrt(81))
print(stats.mean([10, 20, 30]))
print(Path.cwd())
```

9.2 Main guard

```
def main():
    print("Application started")

if __name__ == "__main__":
    main()
```

The main guard runs code only when the file is executed directly, not when it is imported as a module.

9.3 Virtual environment and pip

```
python -m venv .venv
# Windows PowerShell
.\.venv\Scripts\Activate.ps1
python -m pip install requests
python -m pip freeze > requirements.txt
python -m pip install -r requirements.txt
```

Why venv?	Prathi project dependencies separate-ga untayi. Oka project package version vere project ni break cheyyadu.

10. Exceptions and Debugging

10.1 try, except, else, finally

```
try:
    number = int(input("Number: "))
    result = 100 / number
except ValueError:
    print("Enter a valid integer")
except ZeroDivisionError:
    print("Zero is not allowed")
else:
    print(result)
finally:
    print("Finished")
```

10.2 Raise and custom exception

```
class InvalidAgeError(ValueError):
    pass

def register(age):
    if age < 18:
        raise InvalidAgeError("Age must be at least 18")
    return "Registered"
```

Best practice	Bare `except:` errors anni swallow chestundi. Expected specific exceptions catch cheyyi. Fix cheyyaleni exception ni silently ignore cheyyaku.

10.3 Debug checklist

- Traceback bottom line lo exception type and message first chudu.
- Minimal input tho error reproduce cheyyi.
- `print(repr(value), type(value))` or debugger breakpoint use cheyyi.
- Expected vs actual values compare cheyyi.
- Fix tarvatha boundary and invalid inputs test cheyyi.

11. File Handling, CSV and JSON

11.1 Text files with context manager

```
from pathlib import Path

path = Path("notes.txt")
path.write_text("Python revision\n", encoding="utf-8")
text = path.read_text(encoding="utf-8")
print(text)

with path.open("a", encoding="utf-8") as file:
    file.write("Files completed\n")
```

The `with` statement closes the file even if an exception occurs.

11.2 CSV

```
import csv

rows = [
    {"name": "Koushik", "marks": 86},
    {"name": "Meghana", "marks": 91},
]
with open("students.csv", "w", newline="", encoding="utf-8") as file:
    writer = csv.DictWriter(file, fieldnames=["name", "marks"])
    writer.writeheader()
    writer.writerows(rows)
```

11.3 JSON

```
import json

data = {"name": "Koushik", "skills": ["Python", "SQL"]}
with open("profile.json", "w", encoding="utf-8") as file:
    json.dump(data, file, indent=2)
with open("profile.json", encoding="utf-8") as file:
    loaded = json.load(file)
print(loaded["skills"])
```

12. Object-Oriented Programming

12.1 Class, object and constructor

```
class BankAccount:
    bank_name = "Demo Bank"

    def __init__(self, owner: str, balance: float = 0):
        self.owner = owner
        self._balance = float(balance)

    @property
    def balance(self):
        return self._balance

    def deposit(self, amount):
        if amount <= 0:
            raise ValueError("Amount must be positive")
        self._balance += amount

    def __str__(self):
        return f"{self.owner}: {self._balance:.2f}"

account = BankAccount("Koushik", 500)
account.deposit(250)
print(account)
```

Concept	Meaning
Class	Blueprint defining data and behavior
Object	Instance of a class
Encapsulation	Bundle data/behavior and control access by convention/properties
Inheritance	Subclass reuses or extends a base class
Polymorphism	Same interface works with different object types
Abstraction	Expose essential behavior, hide unnecessary implementation

12.2 Inheritance and super

```
class Employee:
    def __init__(self, name):
        self.name = name

    def role(self):
        return "Employee"

class Analyst(Employee):
    def __init__(self, name, tool):
        super().__init__(name)
        self.tool = tool

    def role(self):
        return f"Analyst using {self.tool}"
```

12.3 Class method and static method

```
class Temperature:
    def __init__(self, celsius):
        self.celsius = celsius

    @classmethod
    def from_fahrenheit(cls, value):
        return cls((value - 32) * 5 / 9)

    @staticmethod
    def is_freezing(value):
        return value <= 0
```

self	`self` current object ni represent chestundi. Python automatic-ga method call lo object ni first argument ga pass chestundi.

13. Iterables, Iterators and Generators

13.1 Iterator protocol

An iterable can produce an iterator through `iter()`. An iterator returns values through `next()` and raises StopIteration when finished.

```
numbers = [10, 20, 30]
it = iter(numbers)
print(next(it))
print(next(it))
```

13.2 Generator

```
def countdown(start):
    while start > 0:
        yield start
        start -= 1

for value in countdown(3):
    print(value)
```

A generator yields values lazily, so it can process large sequences without storing all results at once.

13.3 Generator expression

```
total = sum(n * n for n in range(1_000_000))
```

14. Decorators and Context Managers

14.1 Function decorator

```
from functools import wraps

def log_call(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        print(f"Calling {func.__name__}")
        return func(*args, **kwargs)
    return wrapper

@log_call
def add(a, b):
    return a + b

print(add(2, 3))
```

A decorator takes a callable and returns a wrapped callable, adding behavior without changing each function body.

14.2 Custom context manager

```
from contextlib import contextmanager

@contextmanager
def managed_message():
    print("Start")
    try:
        yield
    finally:
        print("End")

with managed_message():
    print("Work")
```

15. Useful Standard Library Tools

Module	Use
pathlib	File and directory paths
datetime	Dates and times
collections	Counter, defaultdict, deque, namedtuple
itertools	Efficient iterator building blocks
functools	reduce, cache, partial, wraps
re	Regular expressions
json / csv	Common data formats
sqlite3	Embedded relational database
logging	Configurable application logs
unittest	Built-in testing framework

15.1 Dates

```
from datetime import date, datetime, timedelta

today = date.today()
next_week = today + timedelta(days=7)
stamp = datetime.now().strftime("%Y-%m-%d %H:%M")
print(today, next_week, stamp)
```

15.2 Regular expressions

```
import re
text = "Order IDs: A-102 and B-450"
ids = re.findall(r"[A-Z]-\d{3}", text)
print(ids)
```

Regex advice	Regex powerful kani normal string methods saripothe avi clearer. Pattern use chesthe raw string `r"..."` prefer cheyyi.

15.3 SQLite

```
import sqlite3

with sqlite3.connect("students.db") as con:
    con.execute("CREATE TABLE IF NOT EXISTS student "
                "(id INTEGER PRIMARY KEY, name TEXT, marks REAL)")
    con.execute("INSERT INTO student(name, marks) VALUES (?, ?)",
                ("Koushik", 86.5))
    rows = con.execute("SELECT name, marks FROM student").fetchall()
print(rows)
```

SQL safety	Values ni f-string tho SQL lo join cheyyaku. `?` placeholders use cheyyi; SQL injection and quoting problems avoid avutayi.

16. Testing, Style and Type Hints

16.1 Assertions and unittest

```
def add(a: int, b: int) -> int:
    return a + b

def test_add():
    assert add(2, 3) == 5
    assert add(-1, 1) == 0

test_add()
```

Assertions are useful for developer checks, but do not use them as the only validation for untrusted user input because optimized runs can remove assert statements.

16.2 Clean-code checklist

- Follow PEP 8: four spaces, meaningful snake_case names and readable line lengths.
- Write small functions with one responsibility.
- Use docstrings for public behavior and comments for why, not obvious what.
- Avoid mutable global state.
- Handle expected errors and validate boundaries.
- Use type hints to improve clarity; Python does not enforce them automatically at runtime.
- Use logging instead of scattered print statements in real applications.

17. Data Analysis Starter: NumPy and pandas

17.1 Install

```
python -m pip install numpy pandas openpyxl
```

17.2 NumPy array

```
import numpy as np

values = np.array([10, 20, 30, 40])
print(values.mean())
print(values * 2)
print(values[values >= 25])
```

NumPy arrays support fast vectorized numeric operations and usually store homogeneous data.

17.3 pandas DataFrame

```
import pandas as pd

df = pd.DataFrame({
    "name": ["A", "B", "C"],
    "sales": [1200, 900, 1500],
    "region": ["South", "North", "South"],
})
print(df.head())
print(df["sales"].sum())
print(df.groupby("region", as_index=False)["sales"].sum())
df.to_excel("sales_report.xlsx", index=False)
```

17.4 Missing values and filtering

```
df = pd.read_csv("sales.csv")
print(df.info())
print(df.isna().sum())
clean = df.dropna(subset=["sales"]).copy()
clean["sales"] = pd.to_numeric(clean["sales"], errors="coerce")
high = clean.loc[clean["sales"] > 1000, ["name", "sales"]]
```

Data analyst habit	Load chesina ventane `head()`, `shape`, `columns`, `dtypes`, `isna().sum()` chudu. Clean chesaka row count and totals validate cheyyi.

18. Advanced Concepts You Should Know

18.1 Shallow vs deep copy

```
import copy
original = [[1, 2], [3, 4]]
shallow = copy.copy(original)
deep = copy.deepcopy(original)
original[0][0] = 99
print(shallow)  # nested list change is visible
print(deep)     # independent nested objects
```

18.2 Memory management and garbage collection

CPython primarily uses reference counting and also has a cyclic garbage collector. `del name` removes a binding; it does not guarantee immediate destruction in every Python implementation.

18.3 GIL

In standard CPython builds, the Global Interpreter Lock allows one thread at a time to execute Python bytecode. Threads can still help I/O-bound tasks. Processes or native/vectorized code often help CPU-bound work. Exact behavior can vary by Python build and version.

18.4 Threading, multiprocessing and async

Tool	Good fit	Important tradeoff
threading	I/O-bound work	Shared-state synchronization
multiprocessing	CPU-bound work	Process startup and data transfer cost
asyncio	Many cooperative I/O operations	Requires async-compatible code

18.5 Time complexity quick view

Operation	Typical average
list append	O(1) amortized
list membership	O(n)
dict/set lookup	O(1) average, O(n) worst case
sorting	O(n log n)
deque append/pop both ends	O(1)

19. Mini Projects

19.1 Number guessing game

```
import random

secret = random.randint(1, 100)
attempts = 0
while True:
    try:
        guess = int(input("Guess 1-100: "))
    except ValueError:
        print("Enter a number")
        continue
    attempts += 1
    if guess < secret:
        print("Too low")
    elif guess > secret:
        print("Too high")
    else:
        print(f"Correct in {attempts} attempts")
        break
```

19.2 Expense tracker

```
import csv
from pathlib import Path

FILE = Path("expenses.csv")

def add_expense(category, amount):
    exists = FILE.exists()
    with FILE.open("a", newline="", encoding="utf-8") as f:
        writer = csv.writer(f)
        if not exists:
            writer.writerow(["category", "amount"])
        writer.writerow([category, amount])

def total_expenses():
    if not FILE.exists():
        return 0.0
    with FILE.open(newline="", encoding="utf-8") as f:
        return sum(float(row["amount"]) for row in csv.DictReader(f))

add_expense("Food", 120)
print(total_expenses())
```

19.3 Project ideas

- CLI to-do app using JSON with add/list/complete/delete.
- Student result analyzer using CSV and pandas.
- File organizer using pathlib and file extensions.
- Quiz game with question bank, score and retry.
- SQLite contact book with CRUD operations.
- Sales dashboard data preparation script for Power BI.

Project explanation	Problem -> input/data -> main functions/classes -> validation -> result -> challenge -> next improvement. Ee order lo 60 seconds answer prepare cheyyi.

20. Python Interview Questions with Model Answers

Q: 1. What is Python?

Answer: Python is a high-level, general-purpose language emphasizing readability, dynamic typing and a large standard/library ecosystem.

Q: 2. Is Python compiled or interpreted?

Answer: Common implementations compile source to bytecode and execute it on a virtual machine. So the simple answer 'only interpreted' misses implementation details.

Q: 3. What is dynamic typing?

Answer: Names are not permanently declared with one type; they can be rebound to objects of different types at runtime.

Q: 4. What is strong typing?

Answer: Python generally does not silently combine incompatible types; for example, a string plus an integer raises TypeError without explicit conversion.

Q: 5. What is indentation in Python?

Answer: Indentation defines code blocks. The standard convention is four spaces per level.

Q: 6. List vs tuple?

Answer: Both are ordered sequences. Lists are mutable; tuples are immutable and can be hashable if all their elements are hashable.

Q: 7. Set vs list?

Answer: A set stores unique hashable elements and offers fast average membership tests. A list preserves positions and duplicates.

Q: 8. Dictionary?

Answer: A mutable mapping from unique hashable keys to values. Modern Python preserves insertion order.

Q: 9. Mutable vs immutable?

Answer: Mutable objects can change in place; immutable objects cannot. This affects sharing, hashing and function behavior.

Q: 10. What is slicing?

Answer: Sequence extraction using start:stop:step, with stop excluded and support for negative indexes/steps.

Q: 11. append vs extend?

Answer: append adds one object as a single list element. extend iterates over its argument and adds each element.

Q: 12. remove vs pop vs del?

Answer: remove deletes the first matching value; pop removes and returns by index; del removes a target or slice and returns nothing.

Q: 13. sort vs sorted?

Answer: list.sort changes the list and returns None. sorted accepts any iterable and returns a new list.

Q: 14. == vs is?

Answer: == checks value equality. is checks whether two references point to the same object.

Q: 15. What are *args and **kwargs?

Answer: *args collects extra positional arguments in a tuple; **kwargs collects extra keyword arguments in a dict.

Q: 16. What is a lambda?

Answer: A small anonymous function limited to one expression. Use def when logic or documentation is substantial.

Q: 17. What is a comprehension?

Answer: Compact syntax for building a collection from an iterable with an expression and optional filters.

Q: 18. What is LEGB?

Answer: Name lookup order: Local, Enclosing, Global and Built-in scopes.

Q: 19. What is a closure?

Answer: An inner function that retains access to variables from its enclosing function even after that outer call returns.

Q: 20. What is a decorator?

Answer: A callable that wraps or transforms another function/class, commonly used for logging, validation and registration.

Q: 21. What is an iterator?

Answer: An object implementing iteration through __iter__ and __next__, raising StopIteration when exhausted.

Q: 22. Iterable vs iterator?

Answer: An iterable can produce an iterator. An iterator maintains traversal state and yields the next item.

Q: 23. What is a generator?

Answer: A lazy iterator commonly created by a function using yield or by a generator expression.

Q: 24. yield vs return?

Answer: return ends a function and returns one result. yield suspends generator state and produces a value, allowing later resumption.

Q: 25. What is exception handling?

Answer: try runs risky code, except handles expected exceptions, else runs on success and finally runs for cleanup.

Q: 26. Why avoid bare except?

Answer: It catches almost everything, including control-flow exceptions, and can hide genuine bugs. Catch specific expected types.

Q: 27. What is a context manager?

Answer: An object or construct managing setup and cleanup around a with block, such as reliably closing files.

Q: 28. What is __init__?

Answer: An initializer called after an instance is created; it configures instance state. It is not the low-level object constructor.

Q: 29. What is self?

Answer: The conventional first instance-method parameter referring to the current instance; it is passed automatically on bound method calls.

Q: 30. Class variable vs instance variable?

Answer: Class variables live on the class and are shared by lookup; instance variables belong to each object's namespace.

Q: 31. Inheritance?

Answer: A class derives behavior/state from a base class and can extend or override it.

Q: 32. Polymorphism?

Answer: Different object types respond to the same operation/interface, often through duck typing or method overriding.

Q: 33. Encapsulation in Python?

Answer: Python uses conventions like _name, name mangling with __name, and properties rather than strict universal private access.

Q: 34. @classmethod vs @staticmethod?

Answer: A class method receives cls and can build/configure class-related objects. A static method receives neither self nor cls automatically.

Q: 35. What is method overriding?

Answer: A subclass supplies its own implementation for a method inherited from a base class.

Q: 36. Module vs package?

Answer: A module is usually one importable Python file; a package groups importable modules under a namespace.

Q: 37. Why use virtual environments?

Answer: They isolate project dependencies and versions, improving reproducibility and preventing conflicts.

Q: 38. What is pip?

Answer: Python's common package installer. Prefer invoking it as `python -m pip` to target the intended interpreter.

Q: 39. Shallow vs deep copy?

Answer: A shallow copy creates a new outer container but shares nested objects. A deep copy recursively copies nested objects when possible.

Q: 40. What is garbage collection?

Answer: Automatic reclamation of unreachable objects. CPython uses reference counting plus a cyclic garbage collector.

Q: 41. What is the GIL?

Answer: In standard CPython, it normally permits one thread at a time to execute Python bytecode; threads still help many I/O-bound workloads.

Q: 42. Thread vs process?

Answer: Threads share process memory and are lightweight; processes have separate memory and can run CPU work in parallel with higher communication cost.

Q: 43. What is async/await?

Answer: Syntax for cooperative asynchronous code, useful for many concurrent I/O operations when libraries support it.

Q: 44. What does if __name__ == '__main__' do?

Answer: It runs a block only when the file is executed directly, not when imported.

Q: 45. What are type hints?

Answer: Optional annotations that improve tooling, documentation and static checks. Python normally does not enforce them at runtime.

Q: 46. What is PEP 8?

Answer: The widely used Python style guide covering formatting, naming and readability conventions.

Q: 47. How do you read a large file?

Answer: Iterate over the file object line by line or process chunks instead of loading the entire file into memory.

Q: 48. How do you handle missing dict keys?

Answer: Use get for a default, membership tests, defaultdict, setdefault or explicit exception handling depending on semantics.

Q: 49. What is enumerate?

Answer: It yields index-value pairs while iterating, avoiding manual index counters.

Q: 50. What is zip?

Answer: It combines iterables item-by-item into tuples and stops at the shortest by default.

Q: 51. Why are dict keys hashable?

Answer: A dictionary uses hash values to locate entries; keys need a stable hash and equality behavior while stored.

Q: 52. What is a dataclass?

Answer: A decorator that generates common methods for data-focused classes, such as __init__ and __repr__, based on annotated fields.

Q: 53. What is monkey patching?

Answer: Changing attributes or behavior at runtime, often in tests; it is flexible but can make code harder to reason about.

Q: 54. How do you debug Python?

Answer: Read the traceback, reproduce minimally, inspect values/types, use breakpoints, add tests and verify edge cases.

Q: 55. Why does a function return None?

Answer: A function without an explicit return, or with bare return, returns None.

Q: 56. What is pandas Series vs DataFrame?

Answer: A Series is one labeled one-dimensional array; a DataFrame is a labeled two-dimensional table of columns.

Q: 57. loc vs iloc in pandas?

Answer: loc selects by labels and boolean masks; iloc selects by integer positions.

Q: 58. How do you handle missing data in pandas?

Answer: Inspect with isna, then choose domain-appropriate drop, fill or model-based treatment and validate the effect.

Q: 59. Why vectorization?

Answer: Vectorized operations express work on whole arrays, often using optimized native code and avoiding slow Python-level loops.

Q: 60. JSON load vs loads?

Answer: load reads JSON from a file-like object; loads parses a JSON string. dump/dumps are the writing counterparts.

21. Output and Coding Interview Practice

21.1 Output questions

```
a = [1, 2]
b = a
b.append(3)
print(a)
```

Answer: `[1, 2, 3]` because a and b refer to the same mutable list.

```
print("Python"[::-2])
```

Answer: `nhy`. Start from end and move by -2 indexes.

```
def add(item, bucket=[]):
    bucket.append(item)
    return bucket
print(add(1))
print(add(2))
```

Answer: `[1]` then `[1, 2]`; the same default list is reused. Use None sentinel.

21.2 Most asked coding tasks

Question	Core approach
Reverse string	s[::-1] or two-pointer explanation
Palindrome	Normalize and compare with reverse/two pointers
Character frequency	collections.Counter or dictionary
Remove duplicates	set if order irrelevant; seen set if order matters
Two sum	Dictionary of value to index, O(n) average
Second largest	Track two distinct values or sort with tradeoff
Merge dictionaries	d1 | d2 in modern Python; later values win
Flatten one level	Nested comprehension or itertools.chain

Model answer - two sum

```
def two_sum(numbers, target):
    seen = {}
    for index, value in enumerate(numbers):
        needed = target - value
        if needed in seen:
            return seen[needed], index
        seen[value] = index
    return None

print(two_sum([2, 7, 11, 15], 9))  # (0, 1)
```

Model answer - second distinct largest

```
def second_largest(numbers):
    largest = second = None
    for value in numbers:
        if largest is None or value > largest:
            if value != largest:
                second = largest
            largest = value
        elif value != largest and (second is None or value > second):
            second = value
    return second
```

Coding interview flow	Requirements clarify -> example -> brute force -> better approach -> code -> dry run -> edge cases -> time/space complexity. Silent-ga code start cheyyaku; thought process short-ga cheppu.

22. Practice Sets and 30-Day Plan

22.1 Beginner practice

- Input name/age and print formatted result.
- Calculator with safe validation.
- Even/odd, leap year, grade and largest of three.
- Factorial, prime, Fibonacci, digit sum and palindrome.
- List min/max/average and duplicates.
- String vowel count, word count and frequency.
- Dictionary-based student marks report.

22.2 Intermediate practice

- CSV expense tracker with category totals.
- JSON to-do list with CRUD actions.
- OOP bank account with validation and history.
- SQLite contact book with parameterized queries.
- Generator that reads a large log file lazily.
- pandas sales cleaning, grouping and Excel export.
- Unit tests for calculator and validation functions.

22.3 30-day schedule

Days	Topics	Deliverable
1-3	Setup, variables, types, I/O	10 small programs
4-6	Operators and conditions	Grade/calculator tasks
7-9	Loops	Digits, prime, Fibonacci
10-12	Strings	Reverse, palindrome, counts
13-16	Collections	List/dict/set problems
17-19	Functions and modules	Reusable utilities
20-22	Exceptions and files	CSV/JSON app
23-25	OOP	Bank/student classes
26	Generators/decorators	2 examples each
27	NumPy/pandas	Sales report
28	Mini project	Complete and test
29	Interview theory	60 answers loud-ga
30	Mock interview	3 output + 2 code + project

Final target	Notes chudakunda core syntax, collections, functions, files, OOP explain chesi 15 common programs rayagaligithe fresher Python interview ki strong base ready.

23. Quick Revision Cheat Sheet

```
name = input("Name: ")
age = int(input("Age: "))
if age >= 18:
    print(f"{name} is eligible")

for i in range(5):
    print(i)

nums = [1, 2, 3]
squares = [x*x for x in nums]
unique = set(nums)
info = {"name": name, "age": age}

def add(a, b=0):
    return a + b

try:
    value = int("10")
except ValueError as exc:
    print(exc)

with open("file.txt", "w", encoding="utf-8") as f:
    f.write("done\n")
```

Need	Use
Index + value	enumerate(items)
Combine lists	zip(a, b)
Safe dict read	d.get(key, default)
Count values	collections.Counter
Queue	collections.deque
Sort by field	sorted(items, key=lambda x: x.field)
File paths	pathlib.Path
Debug value	print(repr(x), type(x))